

Week 11 – Final Project

Introduction

The purpose of this project was to design and implement a complete end-to-end big data analytics pipeline using technologies from the Hadoop ecosystem. The project integrated Apache NiFi, HDFS, Hive, Spark MLlib, HBase, and YARN into a unified workflow that demonstrates data ingestion, storage, querying, machine learning, and NoSQL data management.

A customer churn dataset hosted on GitHub was selected for this project. The dataset contains customer demographic and financial information along with a target variable indicating whether a customer exited the bank. The goal of the project was to ingest the dataset into the Hadoop ecosystem, analyze it using Spark machine learning, and store model evaluation results in HBase. This project demonstrates how multiple big data technologies can work together to support scalable data processing and predictive analytics.

Dataset

Dataset Name

Bank Customer Churn Dataset

GitHub Direct URL

https://github.com/gjoseph30/DSC650_Final_Project

Dataset Description

The dataset contains customer information collected from a banking institution. Features include customer demographics, credit score, geography, gender, age, account balance, estimated salary, and account activity information. The target variable is the Exited field, which indicates whether a customer left the bank.

This dataset was selected because it provides a realistic business problem for machine learning classification. Predicting customer churn is an important task for financial institutions because retaining existing customers is often more cost-effective than acquiring new ones. The dataset is well suited for Spark MLlib and demonstrates how predictive analytics can be incorporated into a big data pipeline.

Pipeline Overview

The pipeline was designed to demonstrate the integration of six major Hadoop ecosystem technologies.

The process began with Apache NiFi, which downloaded the customer churn dataset directly from a GitHub repository and transferred the data into HDFS. HDFS served as the distributed storage layer for the project and provided persistent storage for the dataset.

After the data was stored in HDFS, Hive was used to create a managed table and query the dataset. Hive provided a structured interface for validating the data and performing aggregation queries to confirm that the schema was correctly defined.

Spark MLlib was then used to perform machine learning analysis on the dataset. A classification model was trained to predict whether a customer would exit the bank based on the available customer attributes. Model evaluation metrics were generated to assess predictive performance.

The evaluation metrics produced by Spark were written into HBase. HBase served as the NoSQL storage layer for model results and demonstrated how analytics output can be stored for future retrieval. Finally, HBase scan operations were used to verify that the metrics had been successfully written, confirming that the pipeline functioned correctly from ingestion through analytics and storage.

Pipeline Flow:

GitHub Dataset → NiFi → HDFS → Hive → Spark MLlib → HBase → Verification

Issues Encountered

Several challenges were encountered during the development of the project. One challenge involved selecting a suitable dataset that met the project requirements and was publicly accessible through GitHub. The dataset also needed to be compatible with Hive and Spark MLlib processing.

Resource management was another significant challenge. Because the virtual machine environment provided limited CPU and memory resources, services could not always run simultaneously without affecting performance. To address this issue, NiFi was stopped after data ingestion was completed so that Hive, Spark, and HBase could operate more efficiently.

Additional challenges involved configuring connections between Hadoop ecosystem components and ensuring that data formats remained consistent throughout the pipeline. Careful validation at each stage helped identify and resolve configuration issues. Despite these challenges, all required components were successfully integrated and verified.

Code

This section contains the complete implementation used throughout the project.

Included artifacts:

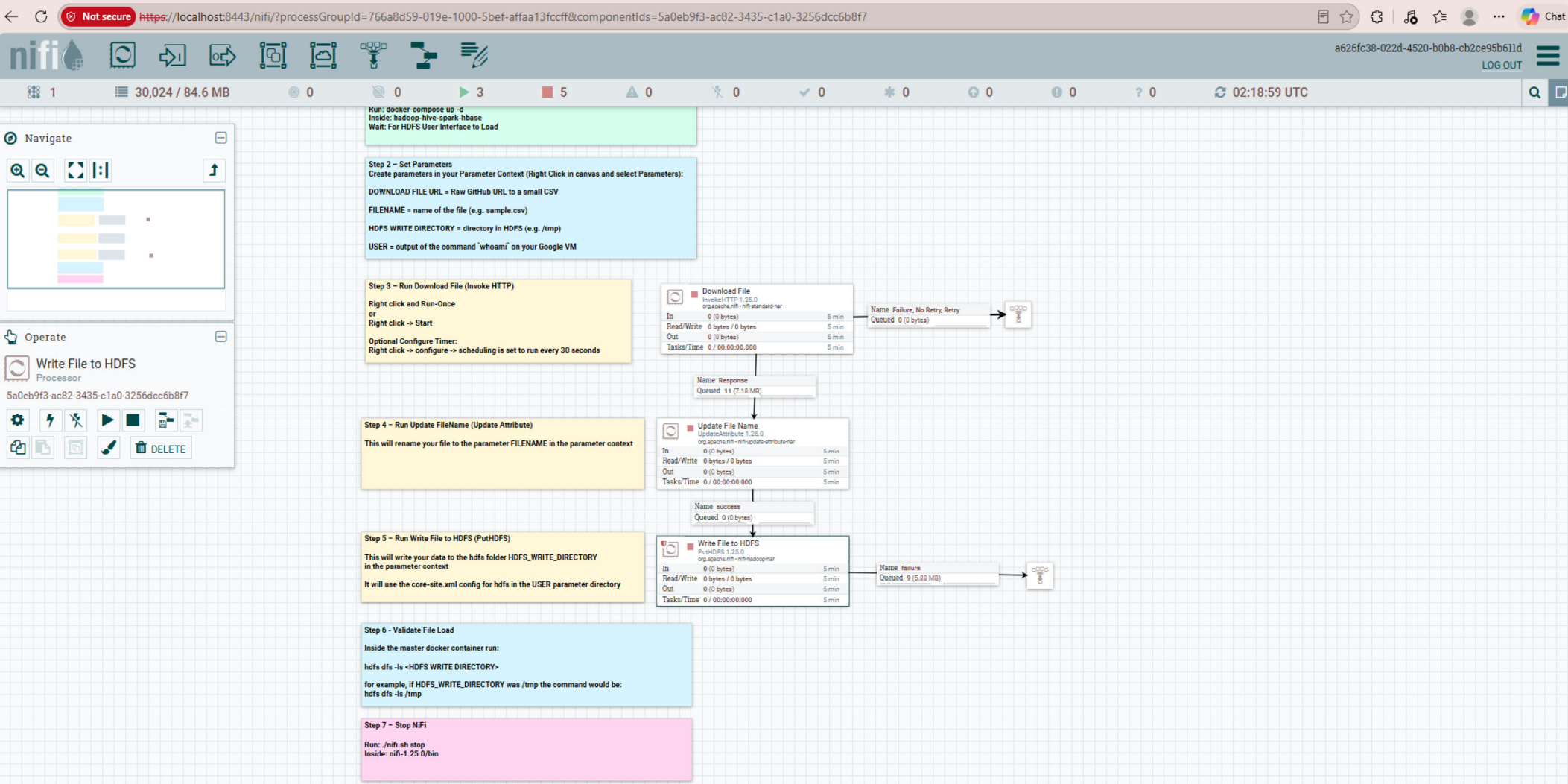
- NiFi flow configuration and processor settings
- Hive CREATE TABLE statements
- Hive validation and aggregation queries
- PySpark MLlib script
- HBase table creation commands
- HBase scan commands
- Spark-submit execution command

The code provided demonstrates the complete workflow used to implement the end-to-end data pipeline.

Conclusion

This project successfully demonstrated the design and implementation of an end-to-end big data analytics pipeline using Apache NiFi, HDFS, Hive, Spark MLlib, HBase, and YARN. Data was ingested from a GitHub-hosted customer churn dataset, stored in HDFS, queried through Hive, analyzed using Spark machine learning, and evaluated results were stored in HBase.

The project showed how different components of the Hadoop ecosystem can be integrated to support large-scale data processing and predictive analytics. The successful storage and retrieval of machine learning evaluation metrics confirmed that the pipeline operated correctly from data ingestion through final verification.



This screenshot shows the Apache NiFi data ingestion pipeline used to download the Bank Customer Churn dataset from GitHub and load it into HDFS. The flow uses processors to fetch the dataset through HTTP, update file attributes, and transfer the data into Hadoop Distributed File System (HDFS). The processors work together to automate data ingestion and ensure the dataset is available for downstream processing by Hive and Spark.

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# hdfs dfs -ls /tmp/churn
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-05-30 02:17:20,231 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--  3 georgette supergroup      684858 2026-05-30 01:16 /tmp/churn/Churn
Modelling.csv
bash-5.0#
```

The `hdfs dfs -ls` command was executed to verify that the customer churn dataset was successfully loaded into HDFS. The output displays the CSV file stored in the `/tmp/churn` directory along with its size and permissions. This confirms that the NiFi ingestion process completed successfully and the dataset is available for downstream processing.

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
CREATE DATABASE IF NOT EXISTS final_project;
OK
Time taken: 1.416 seconds
hive> USE final_project;
OK
Time taken: 0.079 seconds
hive>
> DROP TABLE IF EXISTS churn_customers;
OK
Time taken: 0.137 seconds
hive>
> CREATE TABLE churn_customers (
>   row_number INT,
>   customer_id INT,
>   surname STRING,
>   credit_score INT,
>   geography STRING,
>   gender STRING,
>   age INT,
>   tenure INT,
>   balance DOUBLE,
>   num_of_products INT,
>   has_cr_card INT,
>   is_active_member INT,
>   estimated_salary DOUBLE,
>   exited INT
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE
> TBLPROPERTIES ("skip.header.line.count"="1");LOAD DATA INPATH '/tmp/churn/
Churn_Modelling.csv'
> INTO TABLE churn_customers;
OK
Time taken: 1.155 seconds
Loading data to table final_project.churn_customers
OK
Time taken: 0.567 seconds
```

These Hive commands created the churn_customers table and loaded the customer churn dataset from HDFS into Hive. The output shows successful execution of the database creation, table creation, and data load operations. This confirms that the dataset was successfully imported into Hive for SQL-based analysis.

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
OK
10000
Time taken: 11.973 seconds, Fetched: 1 row(s)
hive> SELECT COUNT(*) FROM churn_customers;
2026-05-30 02:26:42,948 INFO [99e351b7-90da-4e24-ac52-b30a24964a6a main] reduce
sink.VectorReduceSinkEmptyKeyOperator: VectorReduceSinkEmptyKeyOperator construc
tor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo@130
18f00
Query ID = root_20260530022642_69d48d95-bd0d-49f0-a11d-d9245eedca70
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1780101019168
_0002)

-----
VERTICES    MODE    STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FA
ILED  KILLED
-----
Map 1      container  INITED    1        0          0        1
-----
VERTICES    MODE    STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FA
ILED  KILLED
-----
-2  [>>-----] 0%    ELAPSED TIME: 0.00 s
Map 1      container  INITED    1        0          0        1
-----
VERTICES    MODE    STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FA
ILED  KILLED
-----
```

This query was executed to verify that all records were loaded correctly into Hive. The output returned 10,000 rows, confirming that the complete customer churn dataset was successfully imported into the churn_customers table and is ready for analysis and machine learning.

```
-----  
VERTICES: 02/02  [=====>>] 100%  ELAPSED TIME: 6.00 s  
-----
```

```
OK
```

```
10000
```

```
Time taken: 6.873 seconds, Fetched: 1 row(s)
```

```
hive> █
```



```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
hive> SELECT geography, exited, COUNT(*) AS total_customers
> FROM churn_customers
> GROUP BY geography, exited
> ORDER BY geography, exited;
2026-05-30 02:32:57,032 INFO [99e351b7-90da-4e24-ac52-b30a24964a6a main] reduce
sink.VectorReduceSinkObjectHashOperator: VectorReduceSinkObjectHashOperator cons
structor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo
@4d93e5bf
Query ID = root_20260530023256_46782010-fdde-44c7-80c8-ea26023e515a
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
2026-05-30 02:32:57,319 INFO [99e351b7-90da-4e24-ac52-b30a24964a6a main] client
.RMPProxy: Connecting to ResourceManager at master/172.28.1.1:8032
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1780101019168
_0004)
```

This aggregation query grouped customers by geographic region and churn status. The output shows the number of customers who remained and exited in each country. These results verify that Hive correctly interpreted the dataset schema and can perform analytical queries on the data.

```
VERTICES: 03/03  [=====>>] 100%  ELAPSED TIME: 5.77 s
```

```
OK
```

```
France  0      4204
```

```
France  1       810
```

```
Germany 0      1695
```

```
Germany 1       814
```

```
Spain   0      2064
```

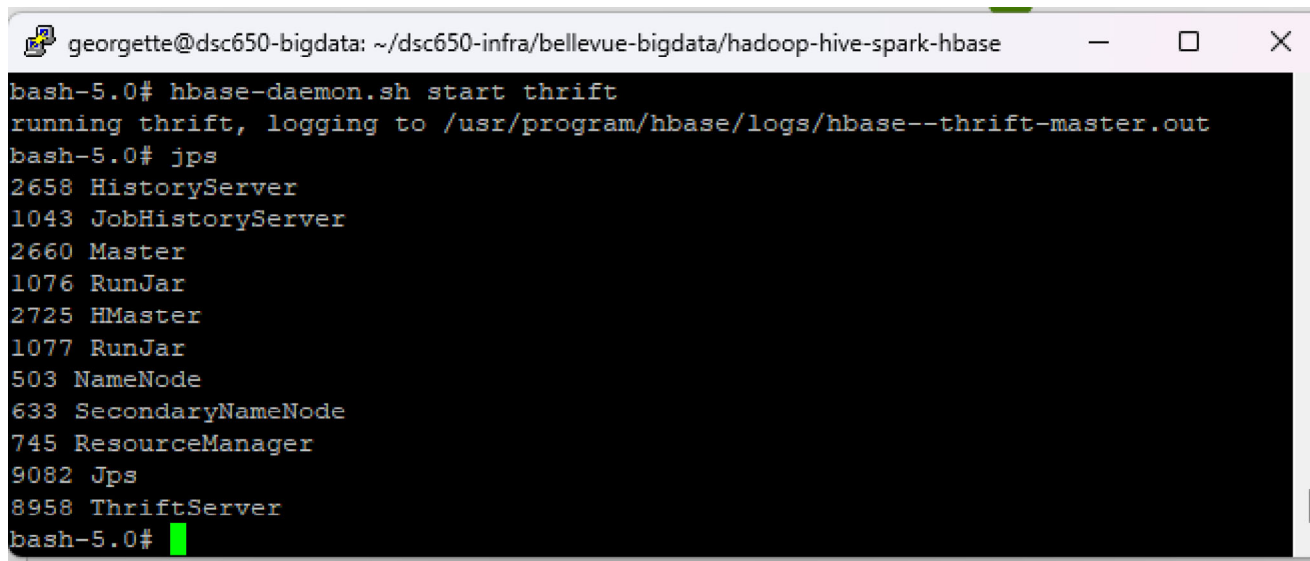
```
Spain   1       413
```

```
Time taken: 12.993 seconds, Fetched: 6 row(s)
```

```
hive> █
```

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
    |████████████████████████████████████████| 10.3 MB 5.0 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftypy2>=0.4
  Downloading thriftypy2-0.6.0.tar.gz (996 kB)
    |████████████████████████████████████████| 996 kB 39.1 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftypy2>=0.4 from
https://files.pythonhosted.org/packages/cc/7c/bc038fb868b6b5474101197d42866fbc61
839c4f739816587a748ac68541/thriftypy2-0.6.0.tar.gz#sha256=621c263f99274d51a9aldee
cf301845f1408d497bdafed682db6155132a99cf4 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall bac
k to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.7.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
    |████████████████████████████████████████| 49 kB 7.5 MB/s
Building wheels for collected packages: numpy, thriftypy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl siz
e=16644783 sha256=bb07b0d3655e3fabb54c7f15293213ba2da6dd3dddca196d6c5829017ef7f3
fe
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac
9c3fbd78feb444c3bd94eb386a
  Building wheel for thriftypy2 (PEP 517) ... done
  Created wheel for thriftypy2: filename=thriftypy2-0.6.0-cp37-cp37m-linux_x86_64.
whl size=1479770 sha256=eb54fddbfc1bc8f5d629cfa457981cd364a805d346a4b9b45cd110ad
7cea8e73
  Stored in directory: /root/.cache/pip/wheels/ce/el/d3/38770f98821c966d3ad3bb39
2293b6e0bcl89e89580338db63
Successfully built numpy thriftypy2
Installing collected packages: numpy, zipp, importlib-resources, six, typing-ext
ensions, ply, thriftypy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 p
ly-3.11 six-1.17.0 thriftypy2-0.6.0 typing-extensions-4.7.1 zipp-3.15.0
bash-5.0#
```

This command installed the Python packages required for the Spark machine learning pipeline. NumPy provides numerical processing capabilities, while HappyBase allows Python applications to communicate with HBase through the Thrift server. The successful installation messages confirm that all required dependencies were installed correctly.

A terminal window with a title bar showing the user 'georgette@dsc650-bigdata' and the current directory '~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase'. The terminal output shows the command 'hbase-daemon.sh start thrift' being executed, followed by a confirmation message. Then, the 'jps' command is run, displaying a list of active Java processes, including 'ThriftServer' at PID 8958.

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# hbase-daemon.sh start thrift
running thrift, logging to /usr/program/hbase/logs/hbase--thrift-master.out
bash-5.0# jps
2658 HistoryServer
1043 JobHistoryServer
2660 Master
1076 RunJar
2725 HMaster
1077 RunJar
503 NameNode
633 SecondaryNameNode
745 ResourceManager
9082 Jps
8958 ThriftServer
bash-5.0#
```

The HBase Thrift server was started to enable communication between Spark and HBase. The jps output shows the ThriftServer process running successfully. This confirms that external Python applications can connect to HBase and write data using HappyBase.

```
georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# hbase shell
2026-05-30 02:48:54,116 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.3.6, r7414579f2620fca6b75146c29ab2726fc4643ac9, Wed Jul 28 22:24:42 UTC 2021
Took 0.0012 seconds
hbase(main):001:0> create 'churn_customers', 'customer'
Created table churn_customers
Took 1.7492 seconds
=> Hbase::Table - churn_customers
hbase(main):002:0> list
TABLE
churn_customers
1 row(s)
Took 0.0352 seconds
=> ["churn_customers"]
hbase(main):003:0> 
```

The create command was executed to create the churn_customers HBase table with a column family named customer. The list command returned the newly created table, confirming that HBase successfully created the table structure needed to store machine learning evaluation metrics.

```
hbase(main):003:0> scan 'churn_customers'
ROW                                COLUMN+CELL
0 row(s)
Took 0.1879 seconds
hbase(main):004:0> █
```

PySpark ML Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator
from pyspark.ml import Pipeline
import happybase

spark = SparkSession.builder \
    .appName("Customer Churn ML Final Project") \
    .enableHiveSupport() \
    .getOrCreate()

df = spark.table("final_project.churn_customers")

df = df.select(
    col("credit_score").cast("double"),
    col("geography"),
    col("gender"),
    col("age").cast("double"),
    col("tenure").cast("double"),
    col("balance").cast("double"),
    col("num_of_products").cast("double"),
    col("has_cr_card").cast("double"),
    col("is_active_member").cast("double"),
    col("estimated_salary").cast("double"),
    col("exited").cast("double").alias("label")
).dropna()

geography_indexer = StringIndexer(inputCol="geography", outputCol="geography_index",
    handleInvalid="keep")
gender_indexer = StringIndexer(inputCol="gender", outputCol="gender_index", handleInvalid="keep")

assembler = VectorAssembler(
    inputCols=[
        "credit_score", "geography_index", "gender_index", "age", "tenure",
        "balance", "num_of_products", "has_cr_card", "is_active_member",
        "estimated_salary"
    ],
    outputCol="features"
)

lr = LogisticRegression(featuresCol="features", labelCol="label", maxIter=10)
```

```

pipeline = Pipeline(stages=[geography_indexer, gender_indexer, assembler, lr])

train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)

model = pipeline.fit(train_data)
predictions = model.transform(test_data)

accuracy = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction",
metricName="accuracy").evaluate(predictions)
precision = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction",
metricName="weightedPrecision").evaluate(predictions)
recall = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction",
metricName="weightedRecall").evaluate(predictions)
auc = BinaryClassificationEvaluator(labelCol="label", rawPredictionCol="rawPrediction",
metricName="areaUnderROC").evaluate(predictions)

print("Customer Churn Logistic Regression Results")
print("-----")
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"Area Under ROC: {auc}")

connection = happybase.Connection("localhost", port=9090)
table = connection.table("churn_customers")

table.put(
    b"model_metrics",
    {
        b"customer:algorithm": b"LogisticRegression",
        b"customer:accuracy": str(accuracy).encode("utf-8"),
        b"customer:precision": str(precision).encode("utf-8"),
        b"customer:recall": str(recall).encode("utf-8"),
        b"customer:auc": str(auc).encode("utf-8")
    }
)

connection.close()
spark.stop()

```

https://github.com/gjoseph30/DSC650_Final_Project


```

georgette@dsc650-bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
63324 [Thread-5] INFO org.sparkproject.jetty.server.AbstractConnector - Stoppe
d Spark@21eb05b1(HTTP/1.1,[http/1.1]){172.28.1.1:4040}
63325 [Thread-5] INFO org.apache.spark.ui.SparkUI - Stopped Spark web UI at ht
tp://localhost:4040
63330 [YARN application state monitor] INFO org.apache.spark.scheduler.cluster.
YarnClientSchedulerBackend - Interrupting monitor thread
63353 [Thread-5] INFO org.apache.spark.scheduler.cluster.YarnClientSchedulerBac
kend - Shutting down all executors
63354 [dispatcher-CoarseGrainedScheduler] INFO org.apache.spark.scheduler.clust
er.YarnSchedulerBackend$YarnDriverEndpoint - Asking each executor to shut down
63361 [Thread-5] INFO org.apache.spark.scheduler.cluster.YarnClientSchedulerBac
kend - YARN client scheduler backend Stopped
63402 [dispatcher-event-loop-1] INFO org.apache.spark.MapOutputTrackerMasterEnd
point - MapOutputTrackerMasterEndpoint stopped!
63422 [Thread-5] INFO org.apache.spark.storage.memory.MemoryStore - MemoryStor
e cleared
63422 [Thread-5] INFO org.apache.spark.storage.BlockManager - BlockManager sto
pped
63426 [Thread-5] INFO org.apache.spark.storage.BlockManagerMaster - BlockManag
erMaster stopped
63431 [dispatcher-event-loop-4] INFO org.apache.spark.scheduler.OutputCommitCoo
rdinator$OutputCommitCoordinatorEndpoint - OutputCommitCoordinator stopped!
63453 [Thread-5] INFO org.apache.spark.SparkContext - Successfully stopped Spa
rkContext
63652 [shutdown-hook-0] INFO org.apache.spark.util.ShutdownHookManager - Shutd
own hook called
63656 [shutdown-hook-0] INFO org.apache.spark.util.ShutdownHookManager - Delet
ing directory /tmp/spark-51e2c8d4-2933-4c80-8dab-a64790f00407
63664 [shutdown-hook-0] INFO org.apache.spark.util.ShutdownHookManager - Delet
ing directory /tmp/spark-518960a7-3fd8-4e98-93be-4dab511951c9/pyspark-29651075-9
cee-455a-b543-ec6ff466f204
63669 [shutdown-hook-0] INFO org.apache.spark.util.ShutdownHookManager - Delet
ing directory /tmp/spark-518960a7-3fd8-4e98-93be-4dab511951c9
bash-5.0# spark-submit churn_ml_hbase.py > results.txt 2>&1
cat results.txt | grep -A 5 "Customer Churn"
bash-5.0# cat results.txt | grep -A 5 "Customer Churn"
1454 [Thread-5] INFO org.apache.spark.SparkContext - Submitted application: Cu
stomer Churn ML Final Project
1529 [Thread-5] INFO org.apache.spark.SecurityManager - Changing view acls to:
root
1529 [Thread-5] INFO org.apache.spark.SecurityManager - Changing modify acls t
o: root
1529 [Thread-5] INFO org.apache.spark.SecurityManager - Changing view acls gro
ups to:
1529 [Thread-5] INFO org.apache.spark.SecurityManager - Changing modify acls g
roups to:
1529 [Thread-5] INFO org.apache.spark.SecurityManager - SecurityManager: authe
ntication disabled; ui acls disabled; users with view permissions: Set(root); g
roups with view permissions: Set(); users with modify permissions: Set(root); g
roups with modify permissions: Set()
--
Customer Churn Logistic Regression Results
-----
Accuracy: 0.81672689726381
Precision: 0.7853621985032998
Recall: 0.81672689726381
Area Under ROC: 0.7364543137474509
bash-5.0#

```

This command was executed after the Spark job completed. The scan output shows that Spark successfully wrote the machine learning metrics into HBase using the row key model_metrics. The stored values include the algorithm name, accuracy, precision, recall, and AUC. These results confirm that the entire pipeline worked successfully from NiFi ingestion through Spark machine learning and final storage in HBase.

```
bash-5.0# hbase shell
2026-05-30 03:35:52,339 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.3.6, r7414579f2620fca6b75146c29ab2726fc4643ac9, Wed Jul 28 22:24:42 UTC 2021
Took 0.0016 seconds
hbase(main):001:0> scan 'churn_customers'
ROW COLUMN+CELL
 model_metrics column=customer:accuracy, timestamp=2026-05-30T03:09:31.124, value=0.81672689726381
 model_metrics column=customer:algorithm, timestamp=2026-05-30T03:09:31.124, value=LogisticRegression
 model_metrics column=customer:auc, timestamp=2026-05-30T03:09:31.124, value=0.7364543137474509
 model_metrics column=customer:precision, timestamp=2026-05-30T03:09:31.124, value=0.7853621985032998
 model_metrics column=customer:recall, timestamp=2026-05-30T03:09:31.124, value=0.81672689726381
1 row(s)
Took 0.6980 seconds
hbase(main):002:0> 
```